# Decomposition vs Augmentation: Isolating the Impact of Multi-Agent Architectures in Entity Resolution

Zachary Zeng
zengz1@ufl.edu
University of Florida
Gainesville, Florida, USA

## Abstract

Recent work reports that multi-agent large language model (LLM) systems outperform single-LLM pipelines on entity resolution (ER), but these systems typically bundle agent decomposition with retrieval-augmented generation or other external knowledge, conflating two design choices. This paper separates them with a three-way comparison on Abt-Buy: a Magellan-style random-forest baseline, a single `gpt-oss-120b` pipeline, and a multi-agent `gpt-oss-120b` pipeline whose syntactic and semantic agents share only local, computable tools under an orchestrator, without external retrieval or task-specific context at inference. Across five runs on one fixed 1,916-pair labeled test split, the single-LLM pipeline attains slightly higher mean $F_1$ than the multi-agent pipeline ($0.9065 \pm 0.0039$ vs. $0.8973 \pm 0.0048$; $\Delta F_1 = +0.0092$, exact permutation test $p = 0.0238$), while the two agree on 98.8% of decisions ($\kappa = 0.9426 \pm 0.0075$). The multi-agent pipeline consumes 8.7× more tokens and runs 6.6× longer, trading recall (0.920 vs. 0.960) for precision (0.875 vs. 0.859). On this benchmark, decomposition without knowledge augmentation acts as a precision-recall adjustment rather than an accuracy gain.

## CCS Concepts

• **Information systems** → **Information extraction**; • **Computing methodologies** → *Knowledge representation and reasoning*; Machine learning approaches; • **Applied computing**;

## Keywords

Entity Resolution, Large Language Models, Multi-Agent Systems, Knowledge Augmentation, Retrieval-Augmented Generation, Agent Decomposition, Zero-shot Classification, Product Matching

**ACM Reference Format:**
Zachary Zeng. 2026. Decomposition vs Augmentation: Isolating the Impact of Multi-Agent Architectures in Entity Resolution. In *Proceedings of ACM SIGKDD Undergraduate Consortium (KDD-UC '26).* ACM, New York, NY, USA, 8 pages. https://doi.org/XXXXXXX.XXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
*KDD-UC '26, Jeju, Korea*
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/08
https://doi.org/XXXXXXX.XXXXXXX

## 1 Introduction

Entity resolution (ER), or record linkage, is the task of identifying which records across one or more datasets refer to the same real-world entity [2]. It is a critical step in data cleaning whose precision directly bounds downstream analysis quality, with applications from census deduplication to academic publication disambiguation [3]. Large language models (LLMs) have recently emerged as a third option alongside rule-based systems and feature-engineered classifiers, leveraging pre-trained world knowledge to assess entity similarity through natural-language reasoning rather than labeled training data and hand-crafted features [12, 14]. This expressiveness, however, comes at a real cost: each pair requires an API call that incurs token-based charges and network latency, so the practical case for LLM-based ER hinges on whether the accuracy gains justify the overhead [7, 10].

A second wave of LLM-based ER work has moved beyond monolithic prompting toward multi-agent architectures, in which specialized agents analyze different aspects of a candidate pair and a coordinator produces the final verdict. Such systems report accuracy gains over single-LLM baselines for ER in Althaf et al. [1], and for the adjacent task of zero-shot named-entity recognition in CMAS [13]. However, they typically bundle agent decomposition with retrieval-augmented generation (RAG), self-annotated training data, or other forms of external knowledge. As a result, the reported gains conflate two distinct design choices: *decomposing* the reasoning across agents, and *augmenting* the agents with information beyond the pair itself. It is not clear whether the multi-agent architecture alone helps ER, or whether the accuracy gains are attributable to the augmentation component and would disappear if the agents were given only the pair and local tools.

This paper isolates that question with a controlled three-way comparison on the Abt-Buy product-catalog benchmark. I evaluated a Magellan-style random-forest baseline [6], a single `gpt-oss-120b` model baseline, and a multi-agent `gpt-oss-120b` pipeline on the same fixed 1,916-pair labeled test split, with identical metrics. The multi-agent pipeline does not include retrieval or external knowledge. Instead, it features a syntactic agent equipped with string-similarity tools (a Ratcliff-Obershelp sequence-matching score, word-token Jaccard, and BM25) and a semantic agent equipped with sentence-embedding cosine similarity and a price comparator. These two agents are coordinated by an orchestrator that merges their verdicts into a final decision. Holding the base model fixed and withholding retrieval from both LLM configurations removes the knowledge-augmentation confound; the multi-agent arm still changes the prompts, call count, tools, and aggregation step at once, so the two LLM arms are read as bundled configurations rather than as a marginal effect of decomposition.

This paper makes three contributions. First, it provides a three-way benchmark between a Magellan-style Random Forest, a single-LLM pipeline, and a multi-agent LLM pipeline on the Abt-Buy benchmark over one fixed labeled test split, with no retrieval or task-specific context given to the LLM pipelines at inference. Second, it reports a cost-efficiency evaluation that includes precision, recall, $F_1$, token consumption, and end-to-end runtime averaged over five separate executions per pipeline, exposing the cost-performance trade-off for practitioners. Third, it presents an error analysis of the pairs in which the two LLM pipelines disagree, categorizing their false positives and false negatives to describe complementary error profiles that accompany the architectural differences.

The results show a small but consistent single-LLM advantage in mean $F_1$ ($0.9065 \pm 0.0039$ vs. $0.8973 \pm 0.0048$; exact permutation $p = 0.0238$), with the two configurations agreeing on 98.8% of decisions. The multi-agent pipeline consumes 8.7× more tokens and runs 6.6× longer while occupying a different point on the precision-recall plane, trading aggressive matches on sparse listings for stricter rejection of color and finish variants.

## 2 Related Work

### 2.1 Entity Resolution Background

Entity resolution has been studied for decades under several names (record linkage, entity matching, deduplication), and modern end-to-end pipelines decompose the task into *blocking*, *matching*, and *clustering* [2, 3]; this paper focuses on the matching stage, evaluating every matcher on one fixed labeled pair set. Early ER relied on Fellegi-Sunter-style probabilistic rules with hand-tuned similarity thresholds [5]; modern supervised approaches such as Magellan [6] (this paper's baseline) automatically generate a dense feature vector of attribute-level similarities (edit distance, Jaccard, Monge-Elkan, TF-IDF cosine) and train a standard classifier on labeled pairs. Deep-learning approaches like DITTO [8] further improved matching by fine-tuning a pre-trained language model on labeled pairs, but they still require task-specific labeled data and per-dataset fine-tuning, motivating the shift toward prompting-based methods.

### 2.2 LLM-Based Entity Resolution

LLM-based ER lets a pre-trained model decide whether two records describe the same entity directly from a natural-language prompt, without task-specific training [5]. Wang et al. [12] identified three prompting strategies: *match* (one pair at a time), *compare* (one record against many), and *select* (pick the best match from a set); this paper's single-LLM baseline uses the *match* strategy as the most direct comparison to traditional pair-classifiers. BoostER complements this design by using the LLM as a selective verifier on top of a cheaper base matcher, reducing the number of LLM calls per dataset [7], and prompt-engineering work has further explored single-attribute prompts and caching to contain token cost [10]. Importantly, recent cross-dataset evaluation shows that fine-tuned small models can rival frontier closed models on entity matching [14], which justifies the use of a medium-sized open model, `gpt-oss-120b`, in this study.

### 2.3 Multi-Agent LLM Systems and the Knowledge-Augmentation Confound

A new paradigm of LLM work uses multi-agent architectures, where specialized agents analyze different aspects of an input and an orchestrator coordinates their verdicts [4]. For ER, Althaf et al. [1] report gains over single-LLM baselines using a layered framework whose agents retrieve context from an indexed knowledge store via a RAG component. Their results show that gains are positive both with and without that RAG layer (single-LLM $F_1$ 86.4, multi-agent without retrieval 88.5, with RAG 93.4), so their own ablation is evidence *against* attributing the whole improvement to retrieval. CMAS [13] pairs decomposition with self-annotated pseudo-labels and retrieved few-shot demonstrations, but it is evaluated on zero-shot named-entity recognition benchmarks rather than on entity resolution. In every reported gain, however, decomposition is coupled with some added source of information beyond the input pair, so the existing literature cannot tell whether the improvements stem from decomposition or from the accompanying knowledge augmentation. This study targets that confound directly: agents have access only to local, computable tools over the pair itself, isolating what decomposition alone contributes to performance and efficiency.

## 3 Method

### 3.1 Data and Blocking

I use the Abt-Buy product dataset from HuggingFace[1]: 1,081 listings from Abt.com and 1,092 from Buy.com, paired into 9,575 labeled candidate pairs with 1,028 verified true matches. Each record carries a name, description, and price (with some missing values), and the dataset is presplit into train (5,743 pairs, 616 positive), validation, and test (each 1,916 pairs, 206 positive) splits, yielding a 10.8% positive-class rate on the test split used for all final metrics.

To reduce the $1{,}081 \times 1{,}092 = 1{,}180{,}452$-pair Cartesian product, I apply TF-IDF cosine-similarity blocking on product names with character 2- and 3-gram features and a similarity threshold of 0.2, retaining 16,629 candidate pairs that contain 100% of true matches.

The blocker characterizes a deployment configuration and is *not* on the evaluated path: every reported metric is computed over the dataset's own 1,916-pair labeled test split, passed directly to each matcher, so all three pipelines see an identical pair set and observed differences are attributable to the matcher.

### 3.2 Magellan Pipeline

The traditional ML baseline is *Magellan-style* rather than a run of Magellan itself: it follows the approach of [6], using a Random Forest classifier over manually engineered features. Because the original `py_entitymatching` library was incompatible with my Python 3.12 environment, I reimplemented the feature set from Magellan's source using `scikit-learn` and `jellyfish` to approximate its auto-generated features. Results for this baseline should therefore be read as a faithful manual adaptation, not as the Magellan library's own output. Each pair is encoded as 14 features: 8 string similarities over names (3-gram and word-token Jaccard, TF-IDF

[1] https://huggingface.co/datasets/matchbench/Abt-Buy

cosine, Monge-Elkan with Jaro-Winkler, raw and normalized Levenshtein, Needleman-Wunsch and Smith-Waterman alignment), 2 over descriptions (3-gram Jaccard, TF-IDF cosine), and 4 over prices. The classifier was trained on the predefined training split, with hyperparameters and the decision threshold tuned by `GridSearchCV` over a `PredefinedSplit` on the validation set to maximize $F_1$ (full grid in the released code).

### 3.3 Single-LLM Pipeline

The single-LLM baseline utilizes the `gpt-oss-120b` model with a direct prompt-response mechanism. Each pair is sent with a system prompt that directs the model to weigh name variations, matching specifications such as model numbers, price compatibility, and description overlap, and to reply with a JSON object holding a binary verdict, a confidence in $[0.0, 1.0]$, and a brief rationale (full prompt in Appendix A). Pairs are processed sequentially at temperature 0, and every response is cached locally for post-hoc analysis.

### 3.4 Multi-Agent LLM Pipeline

The multi-agent LLM architecture decomposes entity resolution into specialized lenses handled by two reviewer agents that are coordinated by a central orchestrator. All agents and the orchestrator use `gpt-oss-120b`, and each agent has a fixed tool-set.

The syntactic agent focuses on string-level similarity and is given three Model Context Protocol (MCP) tools: a normalized sequence-matching dissimilarity over product names, word-token Jaccard similarity over product names, and a symmetric BM25 relevance score normalized to $[0, 1]$.

The sequence-matching tool returns $1 - r$, where $r$ is Python's `difflib.SequenceMatcher` ratio (a Ratcliff-Obershelp measure), so 0.0 denotes identical names; full tool definitions, including the BM25 corpus construction and normalization, are given in Appendix A.

The semantic agent focuses on deeper semantic analysis and is given two MCP tools that it is also instructed to call before deciding: a price comparator that returns the price ratio, absolute difference, and a similarity flag (ratio $\geq$ 0.8) over the two parsed prices, and an embedding cosine similarity computed between `all-MiniLM-L6-v2` sentence-transformer embeddings of the product names.

Tools are served via FastMCP over a stdio transport, running as a subprocess that communicates with the agents through standard input/output, which separates tool execution from LLM inference. The orchestrator receives the original product pair alongside both agents' structured responses (verdict, confidence, and reasoning) and is prompted to review and produce a final decision. Every component (agent or orchestrator) returns a structured JSON response of the form {verdict, confidence, reasoning}, and the orchestrator's verdict serves as the final binary match decision for each candidate pair. Figure 1 illustrates the complete system architecture.

### 3.5 Evaluation Pipeline

All three pipelines share one evaluation harness over the fixed 1,916-pair test split. Each emits a verdict, a confidence, and a parse-error flag (failed JSON parses are recorded, not dropped); tokens are summed across LLM calls per pair, and are zero for Magellan since its inference is local. Each LLM run uses an isolated cache, and Magellan's run-to-run variability comes solely from the Random Forest `random_state`.

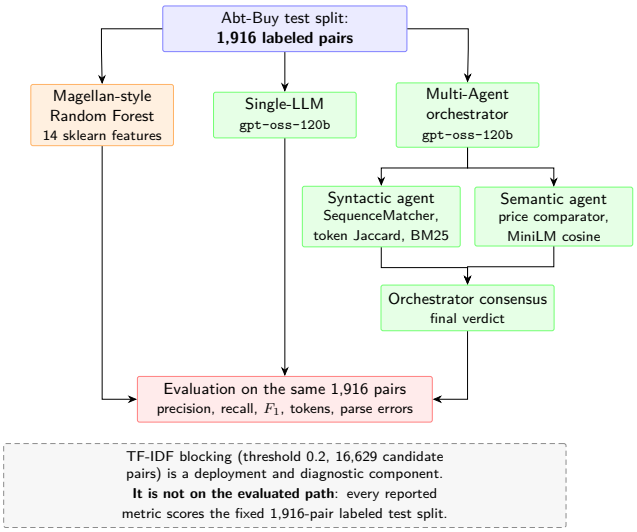


**Figure 1: System architecture. The fixed 1,916-pair labeled test split is passed directly to all three matchers, which are scored by one shared harness. The multi-agent pipeline dispatches each pair to a syntactic and a semantic agent equipped with MCP tools, after which the orchestrator produces the consensus verdict. TF-IDF blocking is shown detached because it characterizes a deployment configuration and did not filter the evaluated pairs.**

## 4 Experiments

### 4.1 Evaluation Metrics

The primary metric is $F_1$, which balances false positives and false negatives without favoring either precision or recall. For the LLM pipelines, I additionally track total tokens consumed and end-to-end runtime, capturing the practical overhead of LLM-based ER relative to traditional ML. The multi-agent pipeline is compared on $F_1$ against both the Magellan and single-LLM baselines, on runtime against both, and on tokens against the single-LLM, to determine whether any performance delta justifies the increased computational cost.

To quantify agreement between the two LLM pipelines beyond chance, I compute Cohen's $\kappa$ over all $5 \times 5 = 25$ pairings of a single-LLM run with a multi-agent run and report the mean and sample standard deviation across those pairings; the configurations were executed independently, so no run-index correspondence exists and any single index-aligned pairing would be arbitrary.

To assess whether the residual $F_1$ gap reflects more than run-to-run noise, the unit of inference is the execution, not the pair: I compare the five run-level $F_1$ values per configuration with an exact, two-sided, unpaired, studentized permutation test over all $\binom{10}{5} = 252$ assignments, which is exact under the null that the ten executions are exchangeable between configurations.

## 4.2 Experiment Setup

All experiments ran on an Apple M4 Pro (24 GB RAM, macOS) with Python 3.12 and uv for dependency management. LLM calls used `gpt-oss-120b` via the OpenAI Python SDK at temperature 0, issued sequentially for comparable runtime measurements. Each pipeline was evaluated on the same 1,916 test-split pairs across 5 separate executions (Magellan variability comes from the Random Forest `random_state`; LLM variability from residual API non-determinism at temperature 0), and I report the mean and standard deviation for all metrics.

## 4.3 Pipeline Performance Comparison

Table 1 presents the aggregate performance of all three pipelines averaged across five separate executions each, treated as independent and exchangeable for inference. The two LLM pipelines reach closely comparable $F_1$ values: the single-LLM baseline achieves $0.9065 \pm 0.0039$ and the multi-agent pipeline achieves $0.8973 \pm 0.0048$. The exact execution-level permutation test resolves this small gap in the single-LLM's favor: $\Delta F_1 = \text{single-LLM} - \text{multi-agent} = +0.0092$, with 6 of the 252 allocations at least as extreme as the observed studentized statistic ($T = 3.355$), giving $p = 6/252 = 0.0238$. The supported claim is narrow: across repeated executions on this fixed Abt-Buy test set, the single-LLM configuration had higher mean $F_1$ than the multi-agent configuration. The entity-cluster sensitivity range, $[-0.0226, +0.0443]$, includes zero: with record- and execution-level uncertainty admitted jointly, the benchmark is compatible with either ordering, so the ranking is a statement about these executions on these pairs. Both LLM pipelines substantially outperform the Magellan-style Random Forest baseline ($F_1 = 0.6066 \pm 0.0161$, roughly 30 points lower); the per-error breakdown is in §4.5.

**Table 1: Aggregate performance across five runs (mean ± sample standard deviation, $n-1$) on the fixed 1,916-pair labeled test split. Accuracy and token metrics derive from the released prediction caches; runtime comes from separate sequential timing executions. The best result in each row is shown in bold; for cost metrics, lower is better.**

| Metric | Magellan | Single-LLM | Multi-Agent |
|---|---|---|---|
| Precision | 0.6876 ± 0.0186 | 0.8585 ± 0.0058 | **0.8754 ± 0.0075** |
| Recall | 0.5427 ± 0.0147 | **0.9602 ± 0.0022** | 0.9204 ± 0.0106 |
| $F_1$ | 0.6066 ± 0.0161 | **0.9065 ± 0.0039** | 0.8973 ± 0.0048 |
| Runtime (s) | **10.0 ± 0.1** | 2,538.9 ± 340.4 | 16,878.5 ± 285.1 |
| Total tokens | **0** | 1,257,041 ± 627 | 10,959,451 ± 24,899 |
| Tokens / pair | **0** | 656.1 ± 0.3 | 5,720.0 ± 13.0 |

Magellan's cost is negligible: about 10 seconds per run and zero tokens, since inference is entirely local. The single-LLM baseline takes about 42 minutes per run and the multi-agent pipeline about 281 minutes (6.6× longer), while also consuming 8.7× more tokens per pair (5,720 vs. 656). The multi-agent pipeline therefore costs substantially more in time and tokens without an accuracy gain, making the single-LLM the more cost-efficient LLM option on this benchmark.

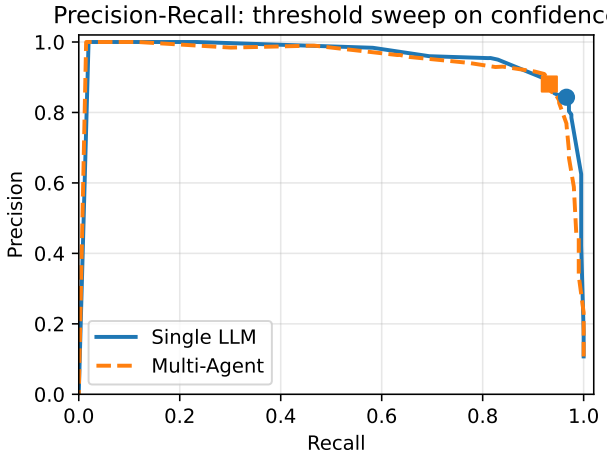


**Figure 2: Precision-Recall curves for the two LLM pipelines on Run 1, obtained by sweeping the decision threshold $\tau \in [0, 1]$ over each pipeline's reported confidence. The reported optima are 0.916 (multi-agent, $\tau = 0.76$) and 0.909 (single-LLM, $\tau = 0.79$). Both pipelines reach 0.906 at the default $\tau = 0.5$, which is the operating point used everywhere else in this paper.**

Despite this cost asymmetry, the two LLM pipelines are closely aligned in their individual decisions: over all 25 pairings of a single-LLM run with a multi-agent run, Cohen's $\kappa$ averages $0.9426 \pm 0.0075$ and decision agreement averages $98.8 \pm 0.15\%$, indicating that multi-agent decomposition reaches largely the same verdicts as the single-LLM at a higher cost.

## 4.4 Precision-Recall Trade-off

Although the two LLM pipelines are tied in $F_1$, they occupy different points on the precision-recall curve, and the direction of the trade-off is informative. Across the five runs, the multi-agent pipeline achieves higher precision ($0.875 \pm 0.008$ vs. $0.859 \pm 0.006$) at the cost of lower recall ($0.920 \pm 0.011$ vs. $0.960 \pm 0.002$). In absolute terms on one illustrative run (Run 1, released as `eval_run_0`, also used for the error analysis below), the single-LLM baseline produces 33 false positives and 8 false negatives, while the multi-agent pipeline produces 26 false positives and 14 false negatives. Run 1 is the multi-agent pipeline's *best* run, so it illustrates the error profiles rather than representing the average case. The multi-agent pipeline is more conservative: it finds fewer positives, but the matches it declares are more consistently correct. Sweeping the decision threshold over each pipeline's reported confidence (Figure 2) shows that this rebalancing persists beyond the default operating point. At the threshold-optimized cut-offs, the multi-agent pipeline reaches its best $F_1 = 0.916$ at $\tau = 0.76$ while the single-LLM reaches $F_1 = 0.909$ at $\tau = 0.79$. The supported finding is qualitative: the two configurations stay close across the threshold range.

## 4.5 Error Analysis

False positives and false negatives from one illustrative run (Run 1) are assigned to categories by deterministic keyword and attribute rules over the product names, descriptions, and each model's reasoning field, in the released `analyze_final_results.py`; results appear in Table 2. The assignment is heuristic and reproducible rather than manually adjudicated. These categories describe patterns in the models' rationales and predictions, not verified mechanisms: self-explanations need not faithfully reflect the computation behind a verdict, and across three LLMs and ten ER datasets Teofili et al. [11] find them unstable and weakly faithful.

**Table 2: False positive and false negative error categories for the two LLM pipelines on one illustrative run (Run 1). Categories are assigned by reproducible keyword and attribute rules over the models' generated rationales; they are descriptive, not verified mechanisms. The lower (better) pipeline total is shown in bold; for error counts lower is better.**

| | Single-LLM | Multi-Agent |
|---|---|---|
| *False positives* | | |
| Color / finish variant | 13 (39%) | 8 (31%) |
| Accessory confusion | 8 (24%) | 6 (23%) |
| Model / SKU variant | 1 (3%) | 3 (12%) |
| Generic vs. specific listing | 1 (3%) | 1 (4%) |
| Form factor | 2 (6%) | 1 (4%) |
| Other | 8 (24%) | 7 (27%) |
| Total FP | 33 | **26** |
| *False negatives* | | |
| Sparse description | 6 (75%) | 11 (79%) |
| Price mismatch | 1 (13%) | 2 (14%) |
| Name mismatch | 1 (13%) | 1 (7%) |
| Total FN | **8** | 14 |

*4.5.1 Single-LLM Errors.* The dominant single-LLM false positive occurs when products share a model family but differ on a discriminating attribute, most often color or finish (13 of 33 false positives, about 39%): for example, matching *Canon PowerShot SD1100 IS silver* to *Canon PowerShot SD1100 IS pink*, where the core model number matches but the suffix denotes a true color variant. The LLM appears to weigh the model-number signal heavily and rationalize the color difference as a minor variant. The single-LLM's false negatives are few (8 total) and concentrated on sparse listings (6 of 8) where one side has no description and there is little evidence beyond the name to confirm a match.

*4.5.2 Multi-Agent LLM Errors.* The multi-agent pipeline produces fewer false positives than the single-LLM (26 vs. 33), with the reduction concentrated where the single-LLM overreached: color variants drop from 13 to 8, and the agents flag more accessory and model-variant cases as distinct. The syntactic agent's SequenceMatcher-based and BM25 tools surface concrete token-level differences in the model suffixes (e.g., *sd1100is* vs. *sd1100is pink*) that the agent cites when rejecting the match. That same conservatism, however, hurts recall on sparse pairs: 11 of the multi-agent pipeline's 14 false negatives involve at least one product listing with an empty or near-empty description, where the only signal available is the short, partially overlapping name, which embeds to moderate cosine similarity and yields low BM25 scores. The single-LLM baseline, lacking any conflicting evidence from strict tools, matches more aggressively on the model number alone in these cases and incurs fewer false negatives as a result.

Per-agent analysis reinforces this asymmetry: the syntactic agent alone reaches $F_1 = 0.9040$ (within 0.002 of the full orchestrated pipeline), while the semantic agent alone reaches only $F_1 = 0.8454$, with 31 false negatives concentrated on the same sparse-description pairs.

*4.5.3 Self-Reported Confidence Implications.* The LLM pipelines also return a self-reported confidence score with each verdict. These are descriptive score patterns, not a validated calibration analysis: no reliability curve, expected-calibration-error, or held-out calibration assessment was performed, so the numbers below describe how the reported scores are distributed across outcome classes and nothing more. The single-LLM's mean reported confidence is lower on its false negatives than its true positives (TP 0.924, FN 0.764, gap 0.161). The multi-agent pipeline shows the same direction with a smaller gap of 0.087. At the agent level the gap is wider than in the orchestrator's output: the syntactic and semantic agents report mean false-negative confidence of 0.731 and 0.704 against true-positive means of 0.823 and 0.865, while the orchestrator's synthesized false-negative confidence is 0.812. Whether that compression costs anything downstream is untested here; establishing it would require the calibration analysis this paper does not perform.

*4.5.4 Magellan Errors.* Magellan's lower $F_1$ is driven by recall (0.5427) more than precision (0.6876): it is reasonably correct when it predicts a match but misses too many. This exceeds the DeepMatcher study's $F_1 = 0.436$ for Magellan on Abt-Buy [9] by about 17 points. The cause was not tested: this reimplementation differs from that study in feature set, hyperparameter and threshold tuning, split, and implementation, and those factors were not separated.

The Magellan feature set encodes string and numeric similarity but cannot represent world knowledge. A *Terk XM mini-tuner home dock, model CNP2000H* and an *Audiovox CNP2000H XM mini-tuner home dock* are the same product sold under two brand names; both LLM pipelines predict `MATCH` at confidence $\geq 0.92$, treating the brand mismatch as retailer labeling over a shared model number, while Magellan predicts `NO MATCH` because the brand-token disagreement outweighs the shared *CNP2000H* identifier.

# 5 Discussion

## 5.1 What Multi-Agent Decomposition Buys and What It Does Not

The central empirical result of this study is that with no external retrieval or task-specific retrieved context available to either LLM configuration, the multi-agent configuration did not improve $F_1$ over a single-LLM baseline: the single-LLM was in fact slightly ahead ($\Delta F_1 = +0.0092$, exact permutation $p = 0.0238$), at roughly one-ninth the token cost. Across all 25 pairings of a single-LLM run with a multi-agent run, the two configurations share a mean Cohen's $\kappa$ of $0.9426 \pm 0.0075$ and agree on 98.8% of decisions. Where they disagree, neither dominates: the direction of the residual error

differs more than its quantity. Decomposition is not useless, though; it shifts the decision boundary consistently toward higher precision and lower recall, and the orchestration layer is implicated in the recall loss. The two agents already agree on 9,316 of the 9,580 decisions across the five runs, and on those the orchestrator overrides their consensus only 21 times (12 correctly). On the 264 cases where the agents disagree, the orchestrator arbitrates correctly 189/264 (71.6%) against 187/264 (70.8%) for the trivial policy of always following the syntactic agent, a near-vacuous margin. Yet 23 of the multi-agent pipeline's 82 false negatives arise in exactly those arbitration cases, so the orchestrator is not a neutral pass-through with respect to recall even though its aggregate arbitration accuracy is unremarkable.

This is a scoping observation about prior work, not a refutation: the studies reporting gains supply information this one withholds (CMAS [13] retrieved demonstrations and type features, Althaf et al. [1] an explicit RAG layer). So, removing augmentation negates the advantage here, but that does not isolate augmentation as the operative component, since those systems differ in more ways than knowledge retrieval. The per-agent breakdown fits the same reading: alone, the semantic agent reaches $F_1 = 0.839 \pm 0.005$ against the syntactic agent's $0.896 \pm 0.005$, weakest on the sparse, overlapping-brand pairs where external knowledge would plausibly help.

## 5.2 LLMs as Zero-Shot Entity Resolvers vs Established Methods

Both LLM pipelines reach $F_1$ near 0.90, well above the Magellan-style baseline ($F_1 = 0.607$) and in the range DITTO reports on this benchmark after task-specific fine-tuning ($F_1 = 0.893$) [8], but with zero labeled training data, no per-dataset feature engineering, and no fine-tuning.

That matters for teams facing ER across many schemas, where traditional and DITTO-style pipelines both need labeled data and per-dataset engineering, while the LLM approach needs only a prompt. The cost is tokens: at roughly \$1-\$15 per million,[2] scoring a deployment-scale 16,629-pair candidate set projects to about \$11-\$164 single-LLM against \$95-\$1,427 multi-agent. Both are far below the labor cost of labeling and maintaining a supervised pipeline, but the multi-agent premium buys no accuracy here.

Both pipelines also report lower confidence on false negatives than true positives (single-LLM 0.764 vs. 0.924). Should that separation survive a proper held-out calibration analysis, it would motivate confidence-guided routing of low-confidence pairs to rules or human review; color and finish variants alone account for ∼35% of false positives (§4.5).

## 5.3 Limitations

**Single benchmark and two-table scope.** All results come from one benchmark: Abt-Buy, a textual, English-language, two-table e-commerce catalog with a 10.8% positive rate on the evaluated split. Nothing here has been tested on lower base rates, longer records such as bibliographic citations (DBLP-Scholar, DBLP-ACM), structured-attribute matching (Amazon-Google, Walmart-Amazon), non-English catalogs, or multi-source settings where schema alignment and transitive closure matter. Those are the natural next evaluations; they were not run, and the direction of the multi-agent effect on them is unknown.

**Single requested model; provider build unrecorded.** Every LLM component here (both reviewer agents, the orchestrator, and the single-LLM baseline) is the same model, `gpt-oss-120b`, at temperature 0. The comparison therefore holds the base model fixed by design, but it also means the result is a statement about one model. A stronger or weaker base model could widen, close, or reverse the gap, and the provider's serving build at run time was not recorded, so exact re-execution is not possible even at temperature 0. In particular, whether a frontier closed-source model would avoid the recall degradation observed under orchestration is untested here. The arbitration analysis in §5 locates much of that loss in the aggregation policy rather than in raw model capability (23 of the pipeline's 82 false negatives arise in arbitration, where the orchestrator beats always following the syntactic agent by only two cases), so a stronger orchestrator model would not automatically remove it, though this remains an empirical question. A further untested variation is heterogeneity: assigning different models to different agents is a common multi-agent design that this single-model setup cannot speak to, since some reported multi-agent gains may depend on capability diversity between agents rather than on decomposition. The midsize open model is justified by evidence that such models rival frontier ones on ER [14]. **Tool design.** The multi-agent tool-set was intentionally restricted to local, computable signals to enforce knowledge isolation, so the findings scope specifically to the no-augmentation regime, not to multi-agent ER in general [13]. **Sequential execution.** Parallelizing the two reviewer-agent calls could reduce runtime without changing tokens or accuracy; the orchestrator still runs after both, and the reduction was not measured.

# 6 Conclusion

In a three-way comparison on Abt-Buy with no external retrieval available to either LLM configuration, the multi-agent LLM pipeline attains slightly lower mean $F_1$ than the single-LLM baseline (0.897 vs. 0.907; exact permutation $p = 0.0238$) while agreeing with it on 98.8% of decisions ($\kappa = 0.943$), at 8.7× the token cost and 6.6× the runtime. Both LLM pipelines clearly outperform the Magellan-style baseline ($F_1 = 0.607$) and reach DITTO's fine-tuned range [8] with no task-specific training. On this test set and under these bundled configurations, the multi-agent architecture bought a precision-for-recall shift rather than an accuracy gain. Prior multi-agent ER work [1] reports gains even without retrieval; this single-benchmark, single-model comparison cannot apportion that difference. Future work should extend this evaluation to multi-source and cross-database ER, mix heterogeneous base models within an agent set, and test confidence-guided routing of uncertain pairs to rules or human review.

## Acknowledgments

This work was completed for CIS6930: Data Engineering at the University of Florida.

[2] Public per-token pricing for mid-tier hosted LLMs spanned roughly this range at the time of writing; these are order-of-magnitude estimates.

## References


[1] Aatif Muhammad Althaf, Muzakkiruddin Ahmed Mohammed, Mariofanna Milanova, John Talburt, and Mert Can Cakmak. 2025. Multi-Agent RAG Framework for Entity Resolution: Advancing Beyond Single-LLM Approaches with Specialized Agent Coordination. *Computers* 14, 12 (2025), 525.

[2] Olivier Binette and Rebecca C Steorts. 2022. (Almost) all of entity resolution. *Science Advances* 8, 12 (2022), eabi8021.

[3] Vassilis Christophides, Vasilis Efthymiou, Themis Palpanas, George Papadakis, and Kostas Stefanidis. 2020. An overview of end-to-end entity resolution for big data. *ACM Computing Surveys (CSUR)* 53, 6 (2020), 1–42.

[4] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680* (2024).

[5] Mahmoud Mohamed Ashour Hussein. 2025. *LLM-Enhanced Entity Matching: Comparative Analysis of traditional and modern techniques.* Master's thesis. ETH Zurich. doi:10.3929/ethz-b-000728906

[6] Pradap Konda, Sanjib Das, Paul Suganthan G. C., AnHai Doan, Adel Ardalan, Jeffrey R. Ballard, Han Li, Fatemah Panahi, Haojun Zhang, Jeff Naughton, Shishir Prasad, Ganesh Krishnan, Rohit Deep, and Vijay Raghavendra. 2016. Magellan: toward building entity matching management systems. *Proc. VLDB Endow.* 9, 12 (2016), 1197–1208. doi:10.14778/2994509.2994535

[7] Huahang Li, Shuangyin Li, Fei Hao, Chen Jason Zhang, Yuanfeng Song, and Lei Chen. 2024. Booster: leveraging large language models for enhancing entity resolution. In *Companion Proceedings of the ACM Web Conference 2024.* 1043–1046.

[8] Yuliang Li, Jinfeng Li, Yoshihiko Suhara, AnHai Doan, and Wang-Chiew Tan. 2020. Deep entity matching with pre-trained language models. *arXiv preprint arXiv:2004.00584* (2020).

[9] Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Rohit Deep, Esteban Arcaute, and Vijay Raghavendra. 2018. Deep learning for entity matching: A design space exploration. In *Proceedings of the 2018 international conference on management of data.* 19–34.

[10] Navapat Nananukul, Khanin Sisaengsuwanchai, and Mayank Kejriwal. 2024. Cost-efficient prompt engineering for unsupervised entity resolution in the product matching domain. *Discover Artificial Intelligence* 4, 1 (2024), 56.

[11] Tommaso Teofili, Donatella Firmani, Nick Koudas, Paolo Merialdo, and Divesh Srivastava. 2026. Can we trust LLM Self-Explanations for Entity Resolution? arXiv:2606.01210 [cs.DB]

[12] Tianshu Wang, Xiaoyang Chen, Hongyu Lin, Xuanang Chen, Xianpei Han, Le Sun, Hao Wang, and Zhenyu Zeng. 2025. Match, compare, or select? an investigation of large language models for entity matching. In *Proceedings of the 31st International Conference on Computational Linguistics.* 96–109.

[13] Zihan Wang, Ziqi Zhao, Yougang Lyu, Zhumin Chen, Maarten de Rijke, and Zhaochun Ren. 2025. A cooperative multi-agent framework for zero-shot named entity recognition. In *Proceedings of the ACM on Web Conference 2025.* 4183–4195.

[14] Zeyu Zhang, Paul Groth, Iacer Calixto, and Sebastian Schelter. 2025. A Deep Dive Into Cross-Dataset Entity Matching with Large and Small Language Models.. In *EDBT.* 922–934.

# A Reproducibility

*Code and Data.* All source code, prompts, and post-hoc analysis scripts are released at https://github.com/Fritozz-105/decomposition-vs-augmentation. The Abt-Buy benchmark is loaded from HuggingFace at https://huggingface.co/datasets/matchbench/Abt-Buy; the predefined `train`/`valid`/`test` splits are used unmodified, and every reported metric is computed on the 1,916-pair `test` split, which is passed directly to each matcher. The five cached prediction runs per LLM pipeline are released under results directory in the source code (the illustrative Run 1 in the text is `eval_run_0`); each holds exactly 1,916 records, and the analysis validates that key set before computing any statistic. The dataset snapshot is not revision-pinned on HuggingFace, so the released caches and the key-label digest recorded in `results/paper_stats.json` are the authoritative identifiers for what was scored.

*Unrecorded Metadata.* Several run-time facts were never captured and are reported as unknown rather than reconstructed: the HuggingFace dataset revision, the serving provider and its build, any request-level seed, and a corroborating record of the evaluation host. The consequence is stated plainly: the reported metrics are fully verifiable from the released caches, but re-running the pipelines is *not* guaranteed to reproduce those caches byte-for-byte, because the provider build behind the API is not identified.

*Environment and Hardware.* Experiments ran on an Apple M4 Pro (24 GB RAM, macOS) with Python 3.12 managed by `uv`. LLM API calls used `gpt-oss-120b` via the OpenAI Python SDK at temperature 0, issued sequentially. The multi-agent pipeline serves its tools via FastMCP over `stdio` transport as a subprocess, with an isolated response cache per run. Magellan run-to-run variability is controlled by the Random Forest `random_state` (varied across the five runs); LLM run-to-run variability comes from residual API non-determinism at temperature 0.

*Commands.* The LLM pipelines' metrics, the agreement and $\kappa$ summaries, both inferential analyses, and the dataset statistics are re-derived from the released caches with *no API calls and no cost*:

```
uv sync --group dev
uv run python -m pytest
uv run python -m scripts.compute_paper_stats
```

Four groups of numbers have separate producers: the Magellan-style metrics (the pipeline itself), the §4.4 threshold optima and per-agent counterfactuals (`analyze_supplementary.py`), the Table 2 categories (`analyze_final_results.py`), and the agent-agreement and arbitration counts (`analyze_arbitration.py`), all computed from the same released caches at no API cost.

The canonical command validates all ten caches, then writes `results/paper_stats.json` (canonical) and `results/paper_stats.md` (rendered from it). Reruns are byte-identical. It reports the per-run confusion matrices, the exact permutation test, the entity-cluster sensitivity range (50,000 replicates, seed `20260730`), and the runtime summary, which it cross-checks against `time.txt` by digest and by re-parsing its durations. Reproducing the predictions themselves *does* incur API cost and requires `evaluate-all -runs 5`; the provider's serving build at our run time was not recorded, so exact re-execution is not guaranteed even at temperature 0.

*MCP Tools.* The multi-agent pipeline exposes five tools. **Sequence dissimilarity** returns $1 - r$, where $r$ is Python's SequenceMatcher from difflib ratio over the two product names. **Jaccard similarity** returns the word-token Jaccard coefficient. **BM25** returns symmetric BM25 relevance over a five-document corpus (the two names plus three empty padding documents, which exist only to give the IDF term enough variance to produce positive scores); the reported value is the mean of the two cross-directional scores, normalized as $s/(s+1)$. **Price comparison** returns the price ratio, absolute difference, and a similarity flag (ratio $\geq 0.8$), with `null` fields when prices are missing. **Embedding cosine** returns the cosine similarity between `all-MiniLM-L6-v2` sentence-transformer embeddings of the two names.

*System Prompts.* All prompts were held constant across the five runs.

*Single-LLM: You are an expert at entity resolution. Determine whether two product listings refer to the same real-world product. Analyze names, descriptions, and prices, considering minor name variations (typos, abbreviations, word order), matching specifications (model numbers, capacities), compatible price ranges, and overlapping descriptions. Respond ONLY with a JSON object:*
`{"verdict": "MATCH"|"NO MATCH", "confidence": 0.0-1.0, "reasoning": ...}`.
*Syntactic agent: You are a syntactic similarity analyst. Call ALL three tools (edit distance, Jaccard, BM25) on the product names, then decide. Respond ONLY with the JSON schema above.*
*Semantic agent: You are a semantic similarity analyst. Call BOTH tools (price comparison, embedding cosine), then decide. Respond ONLY with the JSON schema above.*
*Orchestrator: You are the orchestrator for a multi-agent product entity resolution system. Two specialized agents have independently analyzed the pair. Review their verdicts, confidence scores, and reasoning. You may agree or override their decision if the reasoning is flawed, inconsistent with the evidence, or not justified by the confidence scores. Respond ONLY with the JSON schema above.*

Received 1 May 2026; revised 31 July 2026; accepted 1 September 2026